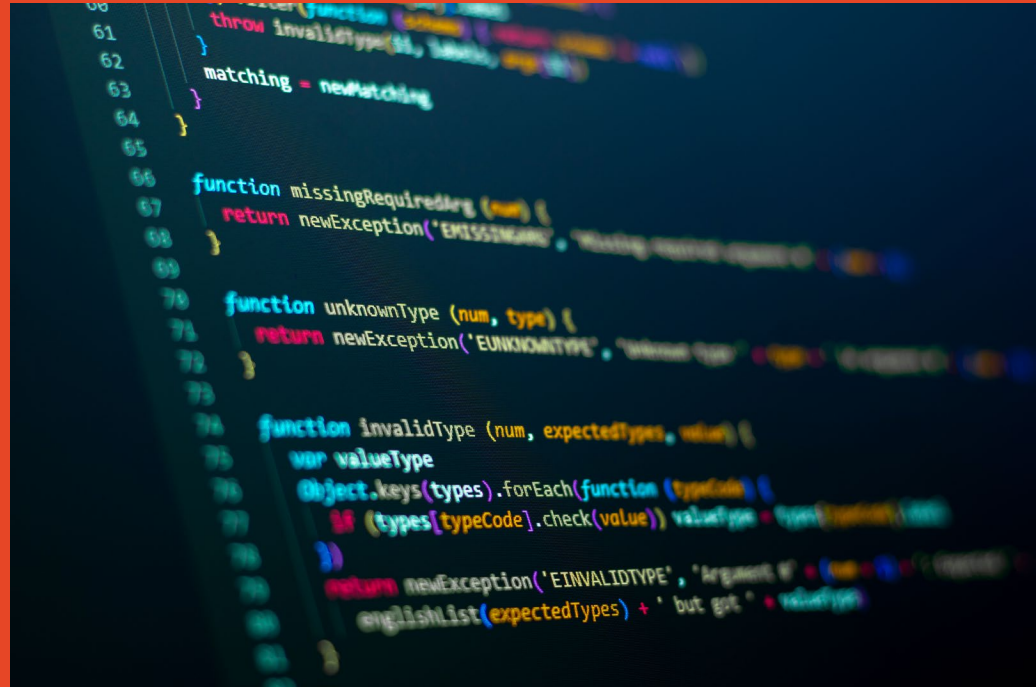


Week 3

Objects, Arrays and Functions



THE UNIVERSITY OF
SYDNEY



This week

- We will learn about:
 - Objects
 - Arrays
 - Functions

Week 2 Recap

- Last week, we looked at primitives.
- This week, we will look at Reference types.
- They are a combination of multiple primitives that create more complex pieces of code.

Objects

Let's go back to Week 1

1: What is Programming?

A program is a recipe for doing something. It has a series of steps that someone reading can follow to perform the described process. Human language is full of ambiguities, and we still do a pretty good job of understanding the steps. For example, the first step here says "boil fresh water in a kettle". This assumes that the person reading knows what "boil" means, or what "fresh water" is. If an alien from another planet came in, they probably would not be able to understand this.

This is what happens with computers—they are incredibly dumb because they don't "know" anything. We have to tell them UNAMBIGUOUSLY what every object means and what we want to do with it.

Programming is the art and science of taking a process that we want executed, and writing it in such a way that the computer can unambiguously execute that process.

What if we want the computer to “know” about real things?

- The way we describe “something” to a computer is through **objects**.
- The best way to understand an object in programming is through examples.
- If we wanted to describe a car, what characteristics should we use?



Cars

- Wheels
- Doors
- Color
- Windows
- Brand



The way we would explain the computer about a car is through objects.

Reference Types: Objects

```
let car = {  
  wheels: 4,  
  doors: 5,  
  color: 'blue',  
  brand: 'Toyota'  
}
```



Reference Types: Objects

```
let car = {  
  wheels: 4,  
  doors: 5,  
  color: 'blue',  
  brand: 'Toyota'  
}
```

We define objects the same way we would define a variable.

Question: What happens if I write “const” instead of “let” to an object?



Reference Types: Objects

```
let car = {  
  wheels: 4,  
  doors: 5,  
  color: 'blue',  
  brand: 'Toyota'  
}
```



The structure of an object is called
Object Literal Notation.

All the information about our class is expressed as “key” and “value” pairs.

Reference Types: Objects

```
let car = {  
  wheels: 4,  
  doors: 5,  
  color: 'blue',  
  brand: 'Toyota'  
}
```



The structure of an object is called **Object Literal Notation**.

All the information about our class is expressed as “key” and “value” pairs.

The keys are the characteristics of the object.

Reference Types: Objects

```
let car = {  
  wheels: 4,  
  doors: 5,  
  color: 'blue',  
  brand: 'Toyota'  
}
```

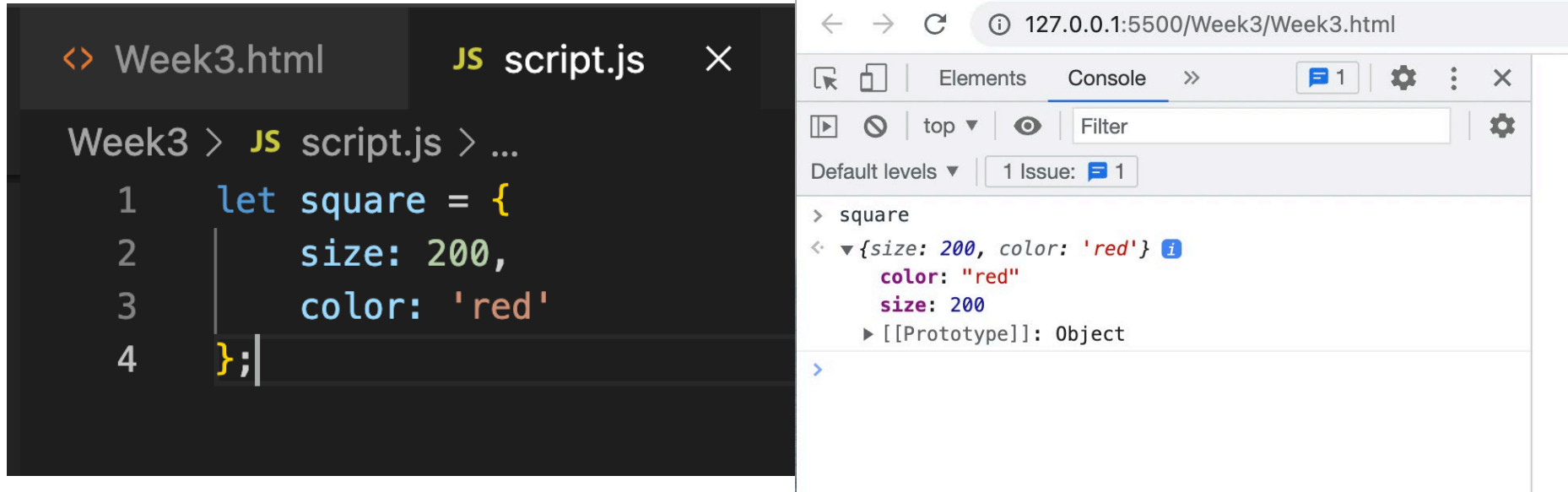


The structure of an object is called **Object Literal Notation**.

All the information about our class is expressed as “key” and “value” pairs.

The values represent the description of each characteristic.

A more topical example of a class



The image shows a web browser window with a code editor and a JavaScript console. The code editor has two tabs: 'Week3.html' and 'script.js'. The 'script.js' tab is active, showing the following code:

```
Week3 > JS script.js > ...  
1 let square = {  
2     size: 200,  
3     color: 'red'  
4 };
```

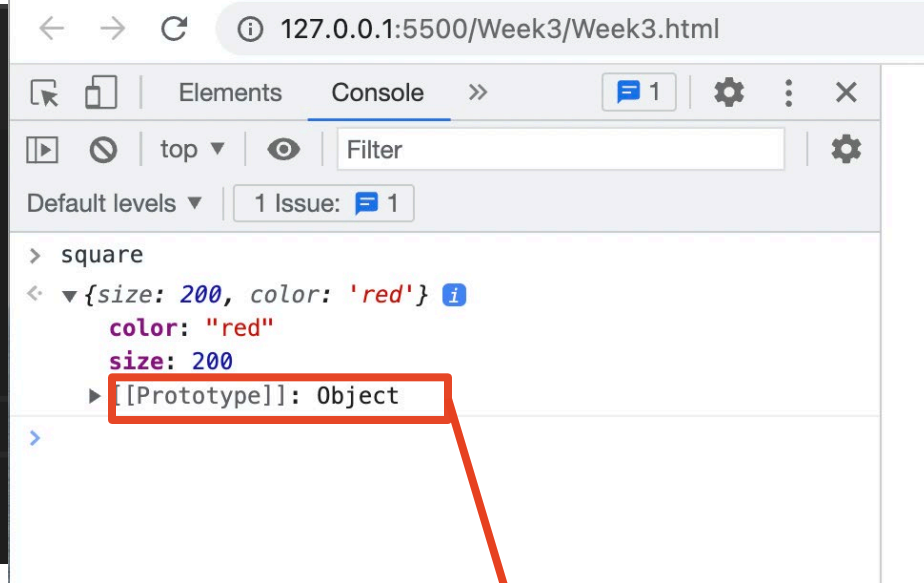
The browser's developer tools are open, showing the 'Console' tab. The console displays the output of the code execution:

```
> square  
{size: 200, color: 'red'}  
  color: "red"  
  size: 200  
  ▶ [[Prototype]]: Object
```

The browser's address bar shows the URL '127.0.0.1:5500/Week3/Week3.html'.

A more topical example of a class

```
Week3 > JS script.js > ...  
1 let square = {  
2   size: 200,  
3   color: 'red'  
4 };
```



JavaScript knows you created an object

Objects: Recap so far

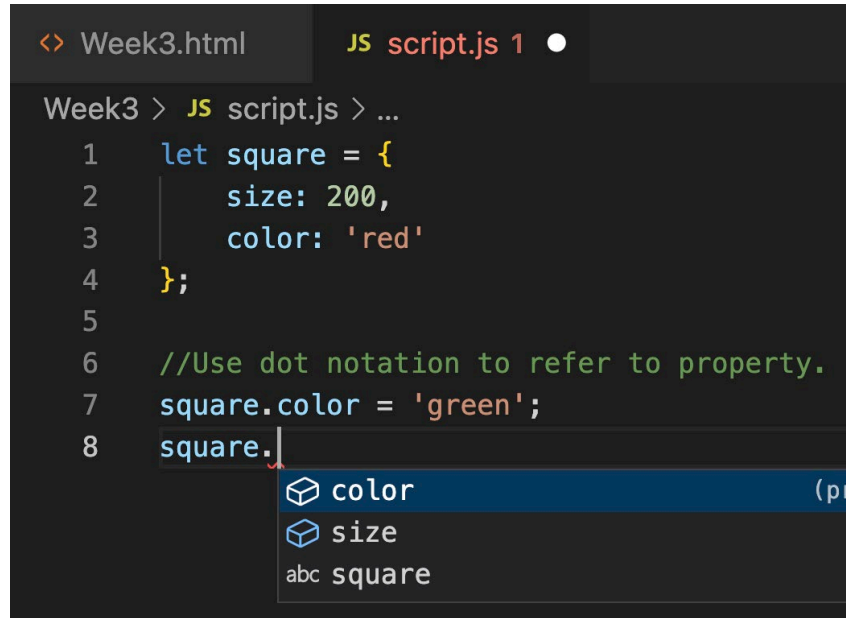
Representation of objects for a computer to understand.

They have attributes and properties, and each of them has different values.

Objects: How do we use objects?

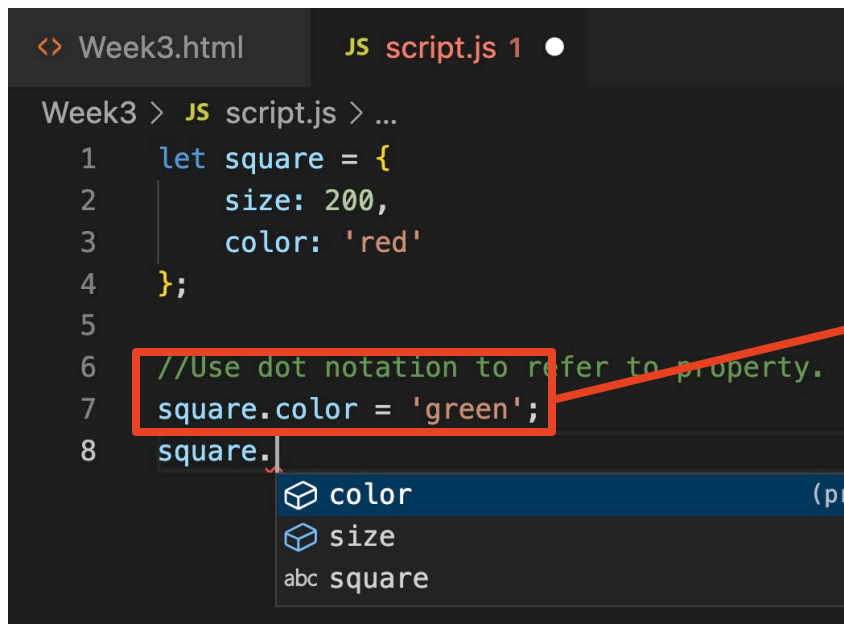
Objects “group” characteristics for us to access and use. One way of accessing these characteristics is through **dot notation**.

```
<> Week3.html JS script.js 1 ●
Week3 > JS script.js > ...
1  let square = {
2      size: 200,
3      color: 'red'
4  };
5
6  //Use dot notation to refer to property.
7  square.color = 'green';
8  square.
```



Objects: How do we use objects?

Objects “group” characteristics for us to access and use. One way of accessing these characteristics is through **dot notation**.



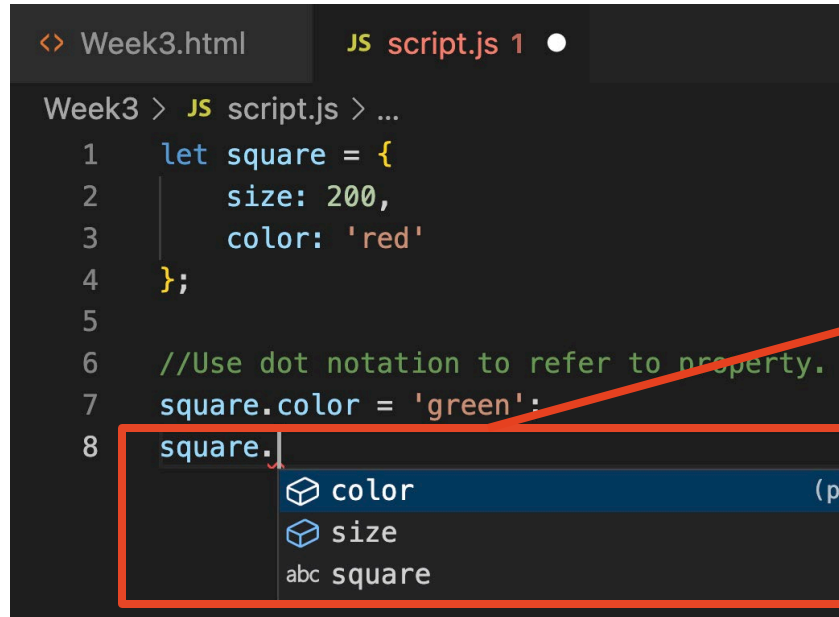
```
<> Week3.html JS script.js 1 ●
Week3 > JS script.js > ...
1 let square = {
2   size: 200,
3   color: 'red'
4 };
5
6 //Use dot notation to refer to property.
7 square.color = 'green';
8 square.
```

color (pr
size
abc square

We can change the values of the properties if we want.

Objects: How do we use objects?

Objects “group” characteristics for us to access and use. One way of accessing these characteristics is through **dot notation**.



The screenshot shows a VS Code editor with two tabs: 'Week3.html' and 'JS script.js 1'. The code in 'script.js' is as follows:

```
Week3 > JS script.js > ...
1  let square = {
2      size: 200,
3      color: 'red'
4  };
5
6  //Use dot notation to refer to property.
7  square.color = 'green';
8  square.
```

At line 8, after 'square.', a dropdown menu is visible, suggesting the following properties:

- color (pr
- size
- abc square

An orange arrow points from the text 'VS Code knows the information about our object and suggests what property to access.' to the dropdown menu.

VS Code knows the information about our object and suggests what property to access.

Objects: How do we use objects?

Another way of accessing these characteristics is through **bracket notation**.

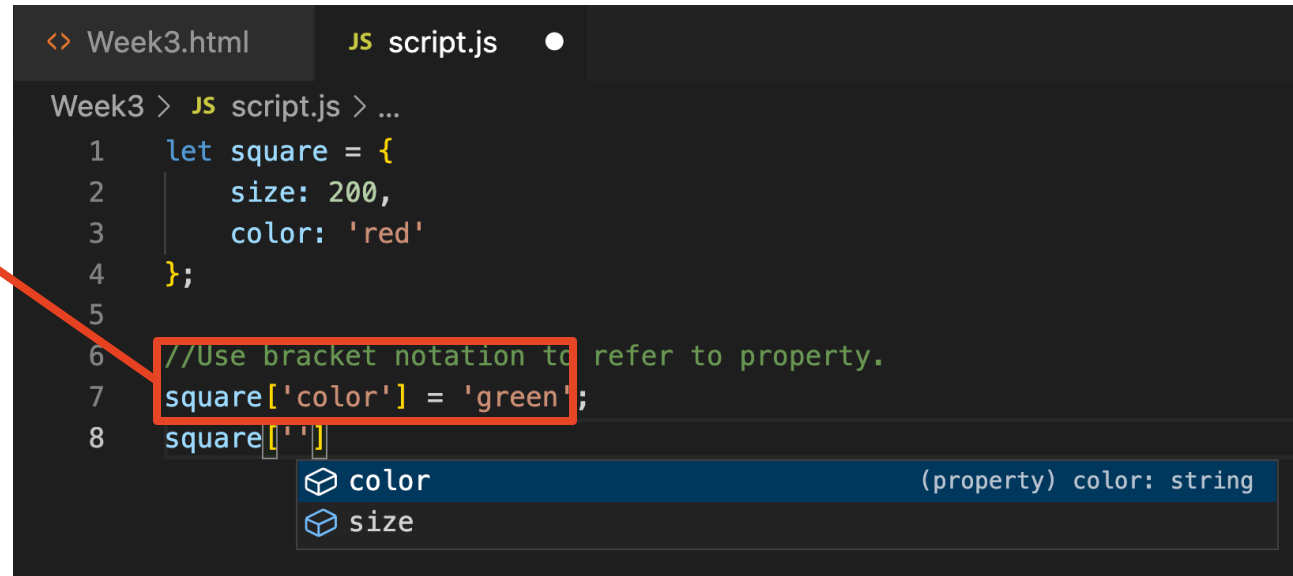
```
<> Week3.html JS script.js ●
Week3 > JS script.js > ...
1  let square = {
2      size: 200,
3      color: 'red'
4  };
5
6  //Use bracket notation to refer to property.
7  square['color'] = 'green';
8  square['']
```

 color (property) color: string
 size

Objects: How do we use objects?

Another way of accessing these characteristics is through **bracket notation**.

Remember to write the property name inside of quotes.



```
<> Week3.html JS script.js •
Week3 > JS script.js > ...
1  let square = {
2      size: 200,
3      color: 'red'
4  };
5
6  //Use bracket notation to refer to property.
7  square['color'] = 'green';
8  square['']
```

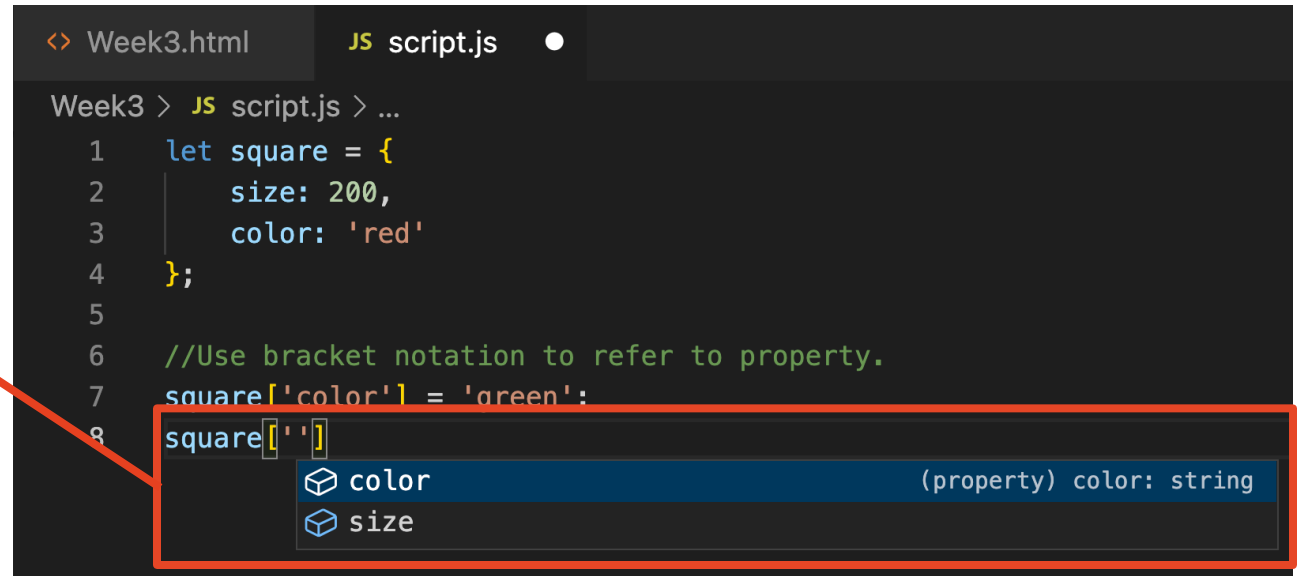
color (property) color: string

size

Objects: How do we use objects?

Another way of accessing these characteristics is through **bracket notation**.

VS Code
ALWAYS
remembers.



```
<> Week3.html JS script.js •
Week3 > JS script.js > ...
1  let square = {
2      size: 200,
3      color: 'red'
4  };
5
6  //Use bracket notation to refer to property.
7  square['color'] = 'green';
8  square['']
```

color (property) color: string
size

Arrays

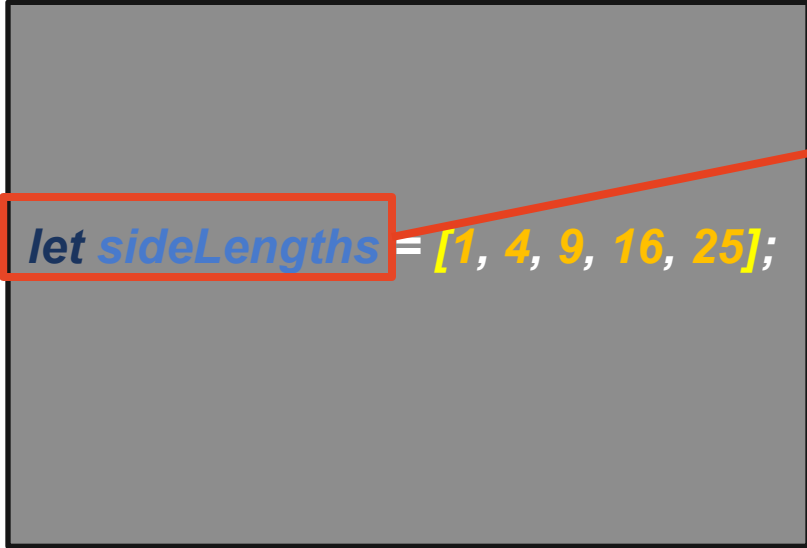
Reference Types: Arrays

- An Array is a list of objects, things, numbers, etc.

```
let sideLengths = [1, 4, 9, 16, 25];
```

Reference Types: Arrays

- An Array is a list of objects, things, numbers, etc.

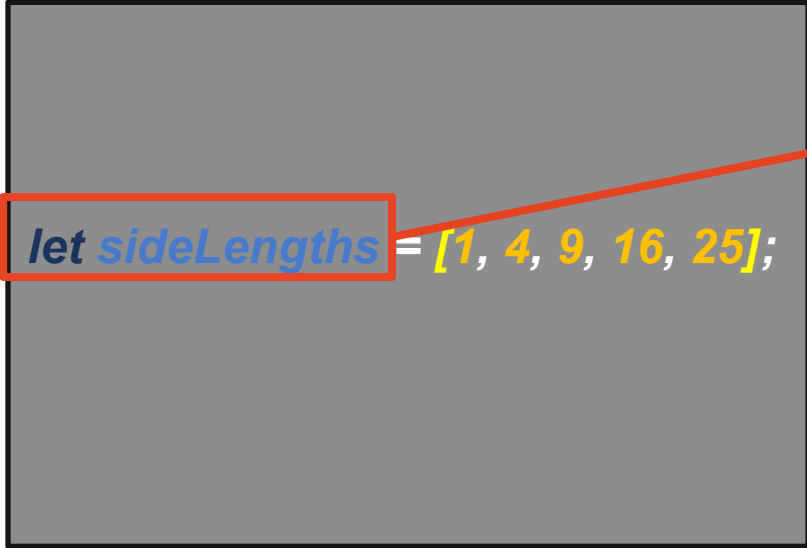


```
let sideLengths = [1, 4, 9, 16, 25];
```

They are declared
the same way you
declare a variable.

Reference Types: Arrays

- An Array is a list of objects, things, numbers, etc.



```
let sideLengths = [1, 4, 9, 16, 25];
```

They are declared
the same way you
declare a variable.

Question: What if we
create a **const** array?

Reference Types: Arrays

- An Array is a list of objects, things, numbers, etc.

```
let sideLengths = [1, 4, 9, 16, 25];
```

The list starts and ends with square brackets []

Reference Types: Arrays

- An Array is a list of objects, things, numbers, etc.

```
let otherList = [1, 5.5, "cow", car, 25];
```

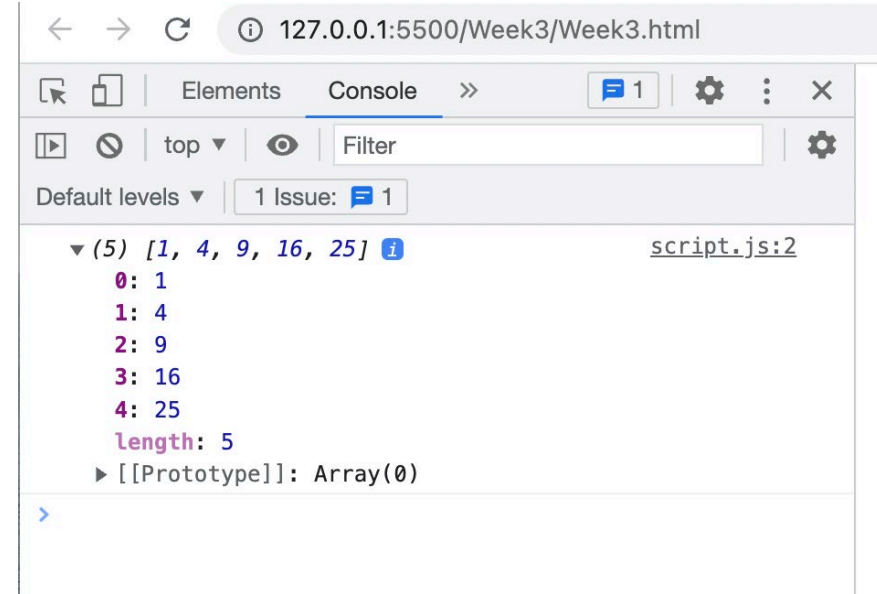
A list can combine multiple types in the same list (other programming languages like C# won't let you do this)

Arrays in the console

<> Week3.html JS script.js X

Week3 > JS script.js > ...

```
1 let sideLengths = [1,4,9,16,25];  
2 console.log(sideLengths);
```



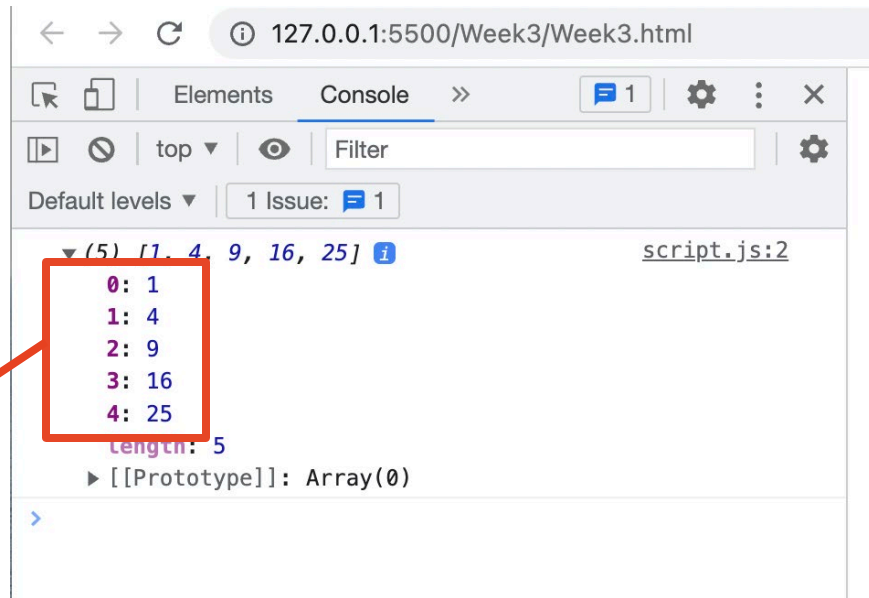
Arrays in the console

<> Week3.html JS script.js X

Week3 > JS script.js > ...

```
1 let sideLengths = [1,4,9,16,25];  
2 console.log(sideLengths);
```

Note how JavaScript tells you the **INDEX** of each element in the array.



Array Indices

- JavaScript starts counting from 0.
- So, the first element of each array will be 0.

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];
```

How to access elements in an array?

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
  
sideLengths [1]
```

How to access elements in an array?

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
sideLengths [1]
```

Note the bracket notation.

How to access elements in an array?

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
sideLengths[1]
```



Inside the brackets,
put the index of the
element you are
accessing.

Array Indices

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
console.log (sideLengths [4]); // what is the output of this?  
// 1? 4? 25?
```

Array Indices

```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
console.log (sideLengths [4]); // The output is 25!
```

Modifying the value in an array

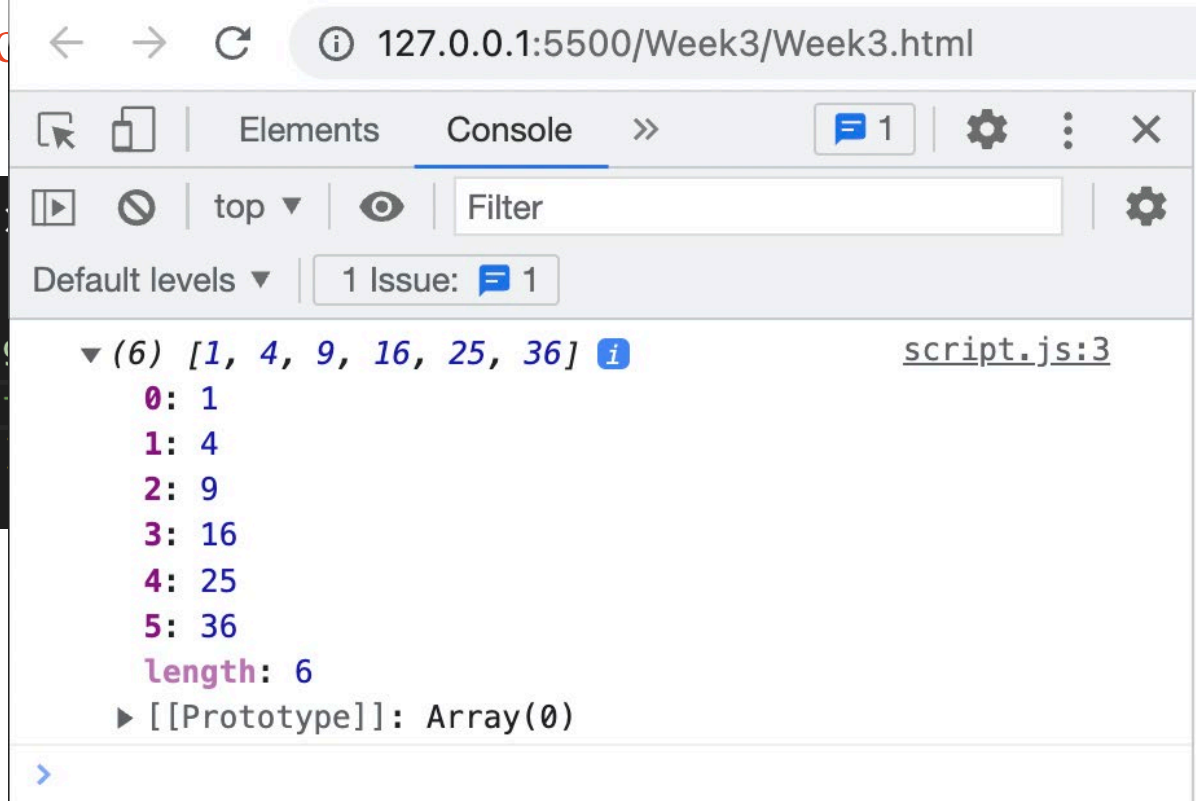
```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
sideLengths [2] = 17;
```

Modifying the value in an array

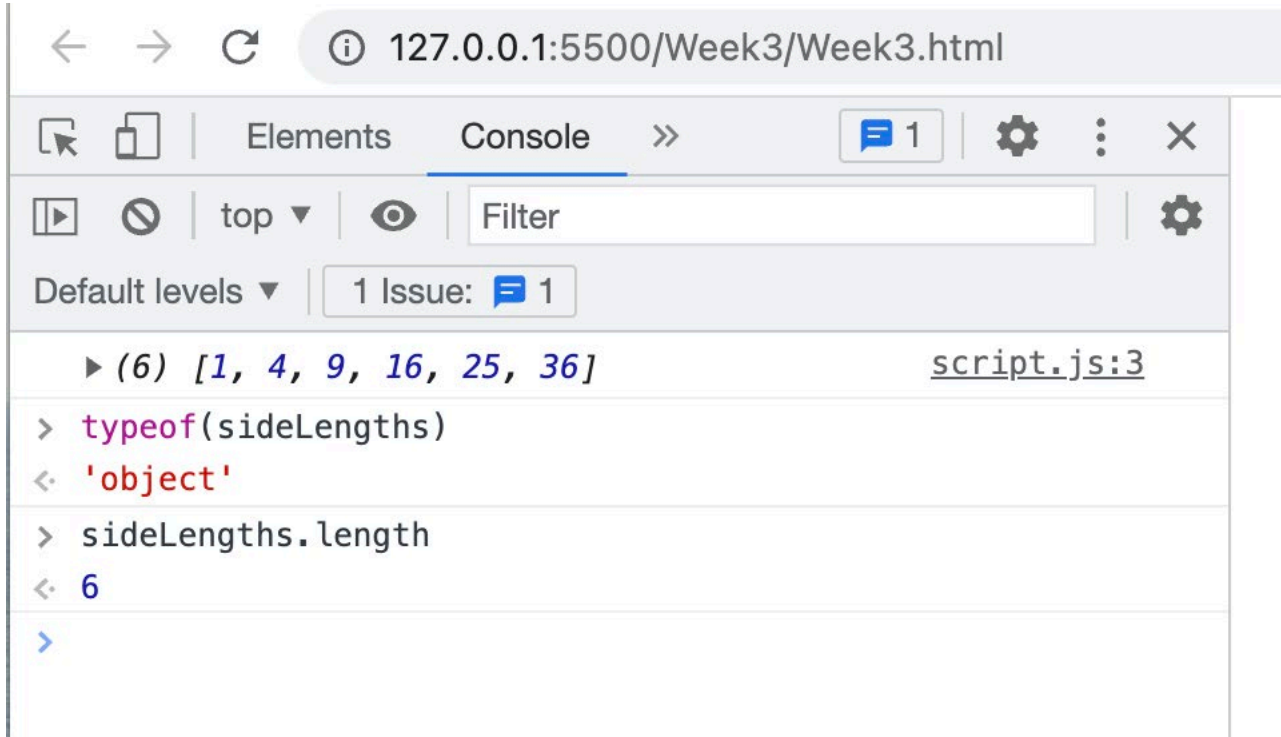
```
// indexes      0, 1, 2, 3 , 4  
let sideLengths = [1, 4, 9, 16, 25];  
sideLengths [2] = 17;  
  
// new configuration is [1, 4, 17, 16, 25]
```

Modifying the length of an array

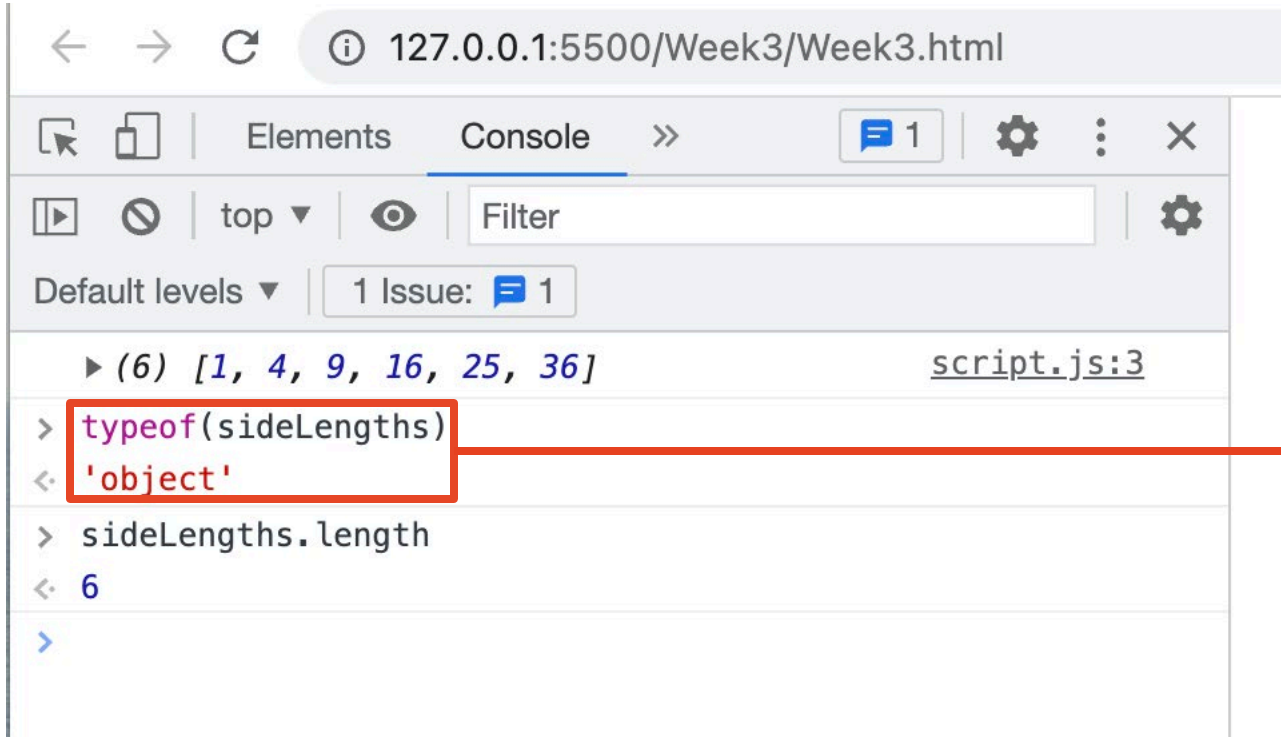
```
<> Week3.html    JS script.js  
Week3 > JS script.js > ...  
1   let sideLengths = [1,4,9,16,25,36];  
2   sideLengths[5] = 36; //  
3   console.log(sideLengths)
```



Arrays as Objects

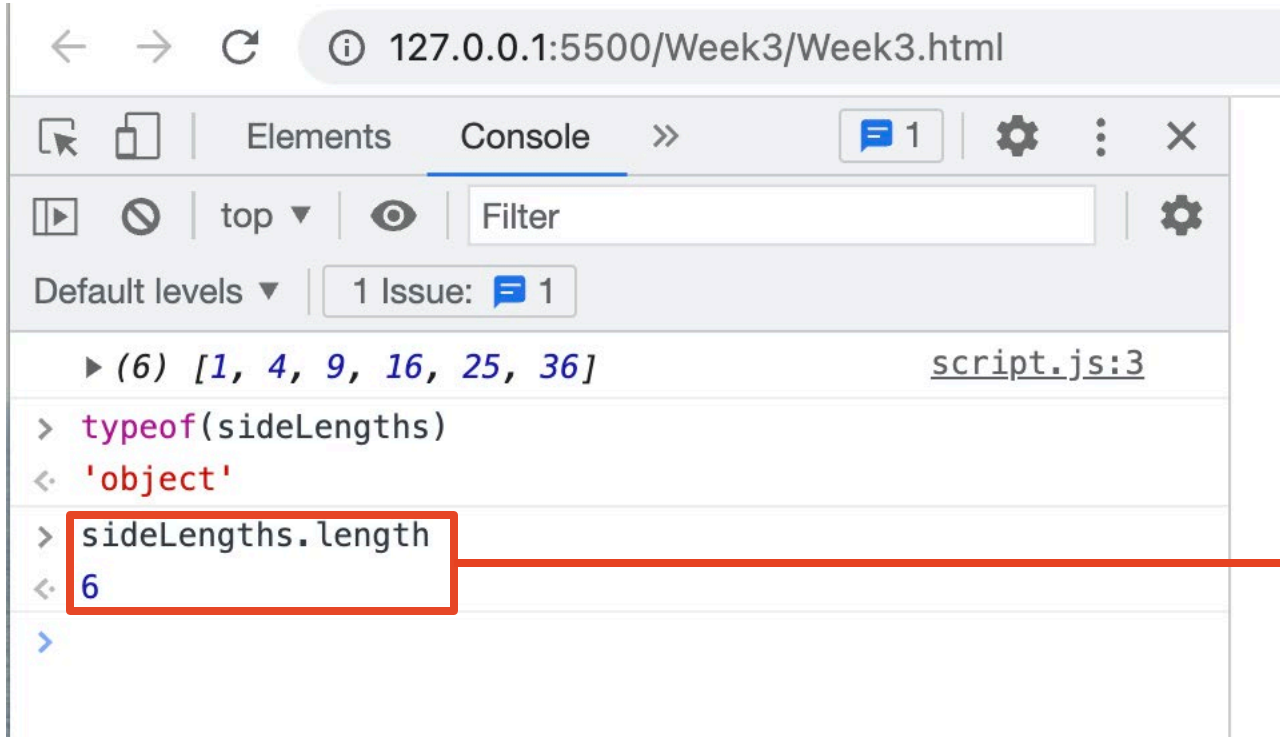


Arrays as Objects



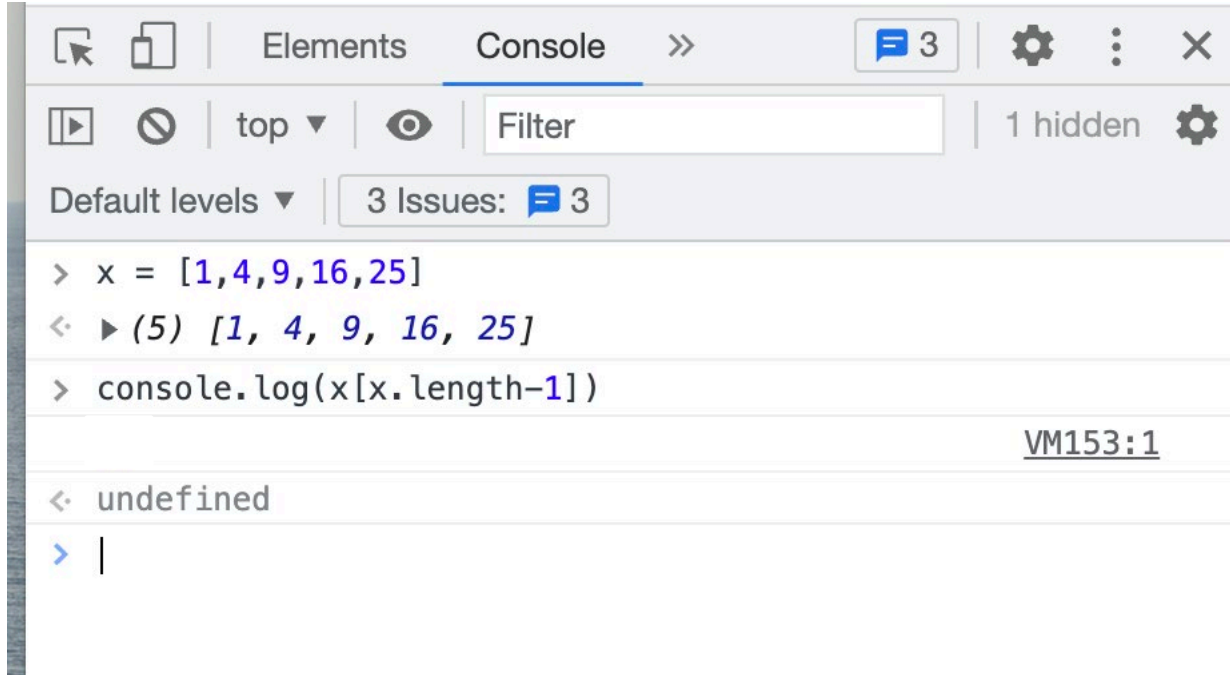
`typeof()` function will return the type **object**. Arrays are an object type provided by JavaScript.

Arrays as Objects

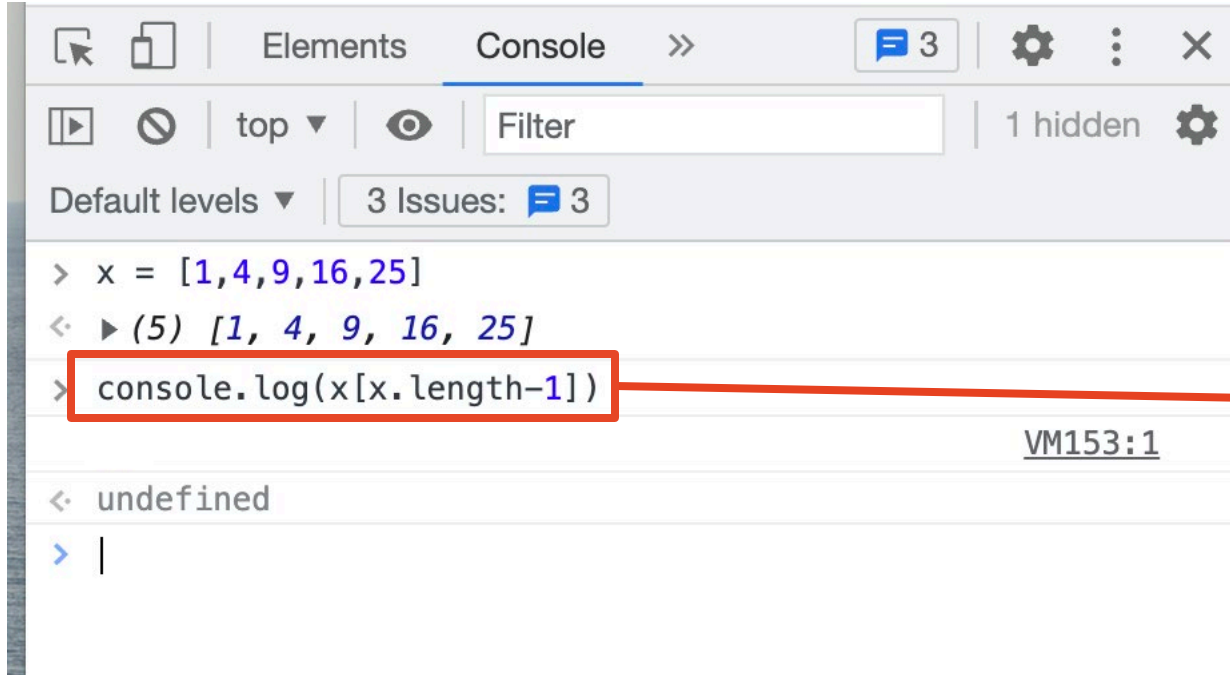


Arrays have the property *length* that tells us how many elements are in the array.

Complex Access to array elements



Complex Access to array elements



The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following code and output:

```
> x = [1,4,9,16,25]
< ▶ (5) [1, 4, 9, 16, 25]
> console.log(x[x.length-1])
VM153:1
< undefined
```

The line `console.log(x[x.length-1])` is highlighted with a red box. A red arrow points from this line to the text 'Question: What will this return?' on the right side of the image.

Question:
What will
this
return?

Complex Access to array elements

```
> x = [1, 4, 9, 16, 25]
<> ▶ (5) [1, 4, 9, 16, 25]
> console.log(x[x.length-1])
25
<> undefined
> |
```

25!

x.length returns 5
x.length - 1 returns 4
x[4] returns 25

You can use
variables as indices
to access array
elements.

More array properties

<> Week3.html

JS script.js 1 ●

Week3 > JS script.js > ...

```
1 let sideLengths = [1,4,9,16,25];  
2 sideLengths[5] = 36; //the sixth element  
3 console.log(sideLengths.);
```

- concat (method) Array<number>.concat(...items: Conc...
- copyWithin
- entries
- every
- fill
- filter
- find
- findIndex
- flat
- flatMap
- forEach
- includes

Functions

Reference Types: Functions

- We learned about the concept of calling functions last week.
- In JavaScript, a function is a collection of programming statements.
- A block of code that takes in some inputs (arguments or parameters), processes those inputs, and outputs desired results.



- Writing functions is one of the most important concepts to learn and master in JavaScript and any programming language you learn.
- A function can have multiple inputs and multiple outputs.

Function Example

```
function sayHello(){  
  // This is the BODY of the function  
  console.log("Hello World Again!");  
  console.log("Gosh, we just keep saying hello in this unit");  
}  
  
sayHello();
```

Function Example

“function” keyword means we are creating a function,



```
function sayHello(){  
  // This is the BODY of the function  
  console.log(“Hello World Again!”);  
  console.log(“Gosh, we just keep saying hello in this unit”);  
}  
  
sayHello();
```

Function Example

“sayHello” is the name of our function (note camelCase)



```
function sayHello(){  
  // This is the BODY of the function  
  console.log(“Hello World Again!”);  
  console.log(“Gosh, we just keep saying hello in this unit”);  
}  
  
sayHello();
```

Function Example

The body of the function has multiple blocks of code.
Note that everything is inside the curly braces.

```
function sayHello(){  
  // This is the BODY of the function  
  console.log("Hello World Again!");  
  console.log("Gosh, we just keep saying hello in this unit");  
}  
  
sayHello();
```

Function Example

```
function sayHello(){  
  // This is the BODY of the function  
  console.log("Hello World Again!");  
  console.log("Gosh, we just keep saying hello in this unit");  
}  
  
sayHello();
```

This is how we call (or use) our function.

Another function example

```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2 ;  
}  
  
console.log (computeArea(42));
```

Another function example

Parameters

Function

Output

```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2 ;  
}  
  
console.log (computeArea(42));
```

Another function example

Parameters

Function

Output

```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2;  
}  
  
computeArea(42);
```

Another function example



```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2;  
}
```

```
computeArea(42);
```

Another function example



```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2 ;  
}  
  
computeArea(42);
```

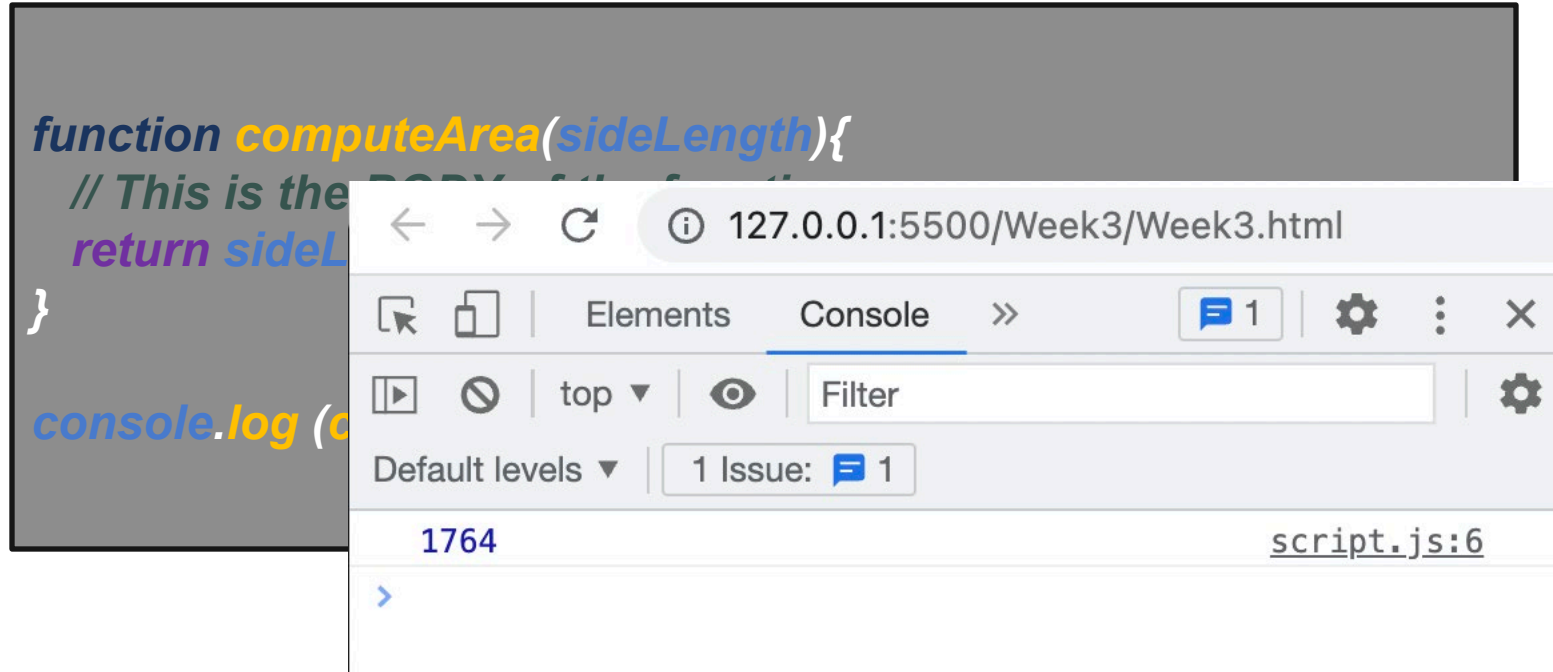
Another function example

Call the function with 42 as an input parameter.

```
function computeArea(sideLength){  
  // This is the BODY of the function  
  return sideLength ** 2 ;  
}
```

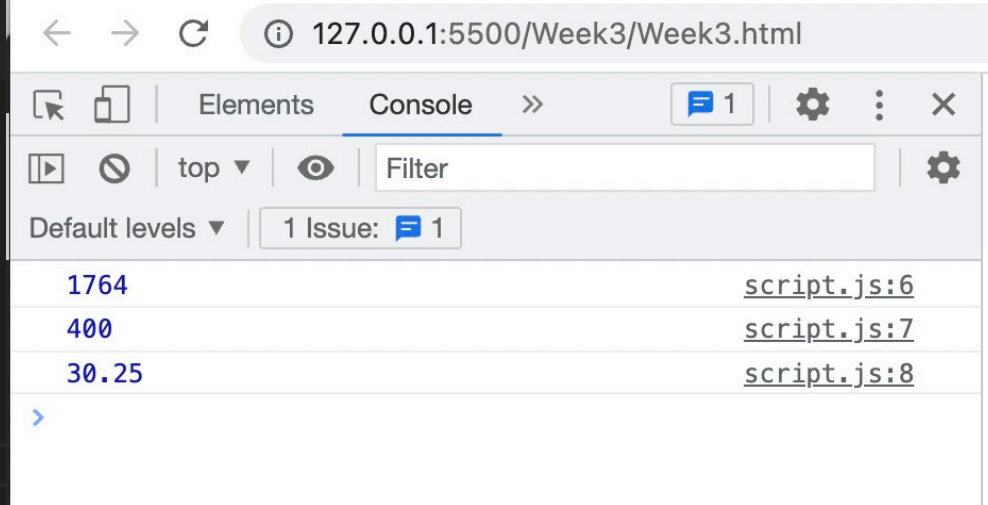
```
computeArea(42);
```

Another function example



Purpose of a function

```
<> Week3.html  JS script.js  x
Week3 > JS script.js > ...
1  function computeArea(sideLength) {
2      //This is the BODY of the function.
3      return sideLength**2;
4  }
5
6  console.log(computeArea(42));
7  console.log(computeArea(20));
8  console.log(computeArea(5.5));
```



Note also the use of nested functions, functions as parameters of functions

Thinking about functions

JavaScript is like a lego set.

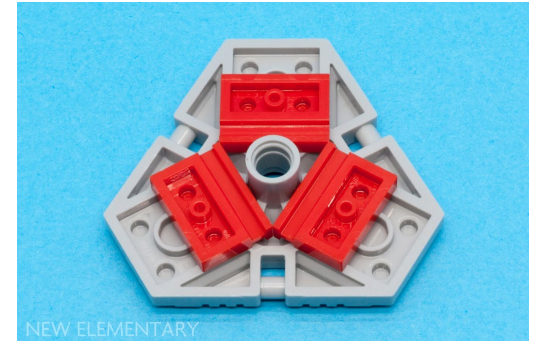
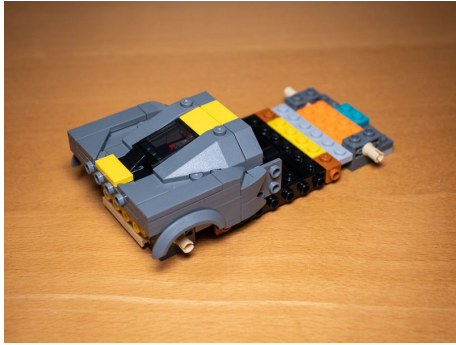


Each block is a primitive or reference type.



Thinking about functions

We put them together to build functions that solve small problems or serve a specific purpose.



Thinking about Functions

We combine them to build a big, visually compelling, artistic piece.



See you in the
tutorial!